

COMPTE RENDU :Simulation de la Diffusion de la Chaleur 2D via une Architecture Hybride MPI / GPU (CuPy)

I. Introduction et Objectifs

L'objectif de ce projet est de concevoir, implémenter et valider un simulateur de l'équation de la chaleur en deux dimensions (2D) capable de passer à l'échelle sur des architectures de calcul à haute performance (HPC).

Pour répondre aux exigences de performances modernes, le projet combine deux paradigmes majeurs :

1. **MPI (Message Passing Interface)** : Pour la distribution des calculs et de la mémoire sur plusieurs nœuds de calcul (parallélisme de processus).
2. **GPU (via la bibliothèque CuPy)** : Pour l'accélération massivement parallèle des calculs matriciels locaux (méthode des différences finies) à l'intérieur de chaque processus.

II. Architecture Technique et Modèle de Parallélisation

1. Topologie de Découpage de la Grille (Domaine Décomposé)

Pour simuler une grille globale de dimensions 120×120 , le domaine spatial a été découpé de manière unidimensionnelle (par blocs de lignes) répartis sur **3 processus MPI** (Ranks 0, 1 et 2).

- chaque sous-grille locale possède une dimension de 40×120 cellules.
- Pour assurer la continuité des calculs aux frontières locales lors de l'application du stencil des différences finies, des lignes de garde (**lignes halos**) ont été implémentées en haut et en bas de chaque sous-grille.

2. Flux de Données Hybride GPU-CPU-MPI

Le défi technique majeur de cette architecture repose sur la gestion de la mémoire entre le GPU (Device) et le CPU (Host) :

- **Calcul local (GPU)** : Les matrices de température sont initialisées et mises à jour à l'aide de **CuPy**, permettant d'exécuter l'algorithme de calcul des différences finies directement sur les cœurs du GPU.
- **Échanges de Halos (MPI sur CPU)** : Les cartes réseau standards et les implémentations MPI classiques nécessitant que les données transitent par la mémoire RAM, les lignes frontières calculées sur le GPU sont temporairement rapatriées sur le CPU via `cp.asnumpy()` avant d'être envoyées par `MPI.COMM_WORLD.Sendrecv`. À la réception, les données de halo sont rechargées sur le GPU.

III. Conditions Initiales et Validation Mathématique

La validation a été réalisée sur un scénario standard de diffusion thermique après **10 itérations temporelles** avec un coefficient de diffusion $\alpha = 0.1$.

1. Configuration Initiale

- **Grille globale** : 120×120 (soit 14 400 mailles).
- **Source de chaleur** : Une perturbation thermique localisée de 100.0°C a été appliquée sur un bloc central de 6×40 mailles géré spécifiquement par le processus central (Rank 1).
- **Reste de la grille** : Initialisé à 0.0°C .

2. Analyse des Résultats de Validation (Rank 0)

Les logs de l'infrastructure de validation exécutée sur le nœud maître (Rank 0) démontrent la rigueur mathématique du code :

- **Reconstruction globale** : La méthode `gather_grid` a reconstruit sans erreur la matrice complète en préservant le format strict (120,120).
- **Température Maximale (95.66°C)** : Le pic initial de 100.0°C s'est dissipé de manière fluide et continue vers les mailles adjacentes, confirmant que le stencil physique et les transferts de halos entre processus fonctionnent de manière synchrone et correcte.

- **Température Moyenne Globale (1.6667°C) :** Ce résultat confirme de manière stricte le **principe de conservation de l'énergie**. La quantité totale de chaleur injectée au départ ($240 \text{ mailles} \times 100^\circ\text{C} = 24\,000$) divisée par la surface totale (14 400 mailles) équivaut exactement à 1.6666...° C. Aucune perte ou gain de chaleur numérique n'est à déplorer aux interfaces de communication.

IV. Forces, Limites et Perspectives

Points Forts

- **Stabilité et Précision :** Conservation parfaite de l'énergie thermique à travers les communications inter-processus.
- **Modularité :** Structure de code propre permettant d'isoler le calcul purement matriciel GPU de la couche de transport réseau MPI.

Goulot d'Étranglement Identifié (Bottleneck)

L'obligation d'effectuer des conversions explicites GPU $\rightarrow$ CPU (cp.asnumpy()) pour les fonctions de communication MPI, puis CPU $\rightarrow$ GPU à la réception, génère un coût de transfert (overhead) important sur le bus PCIe. Pour de très grandes grilles, ce transfert mémoire bride la puissance de calcul brute des GPU.